

Gemini Intro

- IN this project we will create a API key and use it to build a model. - Steps to get API key: Get API Key: Google Deepmind-> Gemini-> try google in AI studio->get api key->create api key. - To understand and generate content, will use gemini-2.5-flash-preview-04-17 generative model. - This model can be used to perform different task like 1. Text Generation: Writing article,poem,answering question,translation and summurizing content. 2. Image Input(Multimodel): Describing image,reading text from image(OCR-optical character recognition) 3. Code Help: Explaining,debugging code 4. Resoning and Math: Solviving logical and math problem 5. Chat like Conversation: Can create Chat session and maintain like a chatbot

```
In [2]: import os           # to access computer system,like reading files and fetching envi
```

```
In [3]: os.environ["GEMINI_API_KEY"] = "AIzaSyBn2hqWwupEpFatVyQ4cvnR5A2l50mzxz8"      # st
```

```
In [4]: !pip install -q -U google-generativeai # install/ update generativeai lebrary of g
```

```
ERROR: Invalid requirement: '#': Expected package name at the start of dependency sp
ecifier
#
^
```

```
In [5]: import google.generativeai as genai    # import library as genai for easy reference
```

```
In [6]: genai.configure(api_key=os.environ["GEMINI_API_KEY"])    # configure secret key wit
# with this step gemini knows who is using it's services
```

```
In [7]: model = genai.GenerativeModel("gemini-2.5-flash-preview-04-17")    # Load mode
```

Text Generation(answering question)

```
In [9]: response = model.generate_content("does agentic ai is the future")
print(response.text)# ask question, get response
```

Yes, **agentic AI** is widely considered a significant and likely future direction for artificial intelligence.

Here's why:

1. **Evolution beyond reactive models:** Current AI models (like basic chatbots) are often reactive – they respond to a single prompt or turn. Agentic AI aims to create systems that are proactive, goal-oriented, and can perform multi-step tasks autonomously or semi-autonomously.
2. **Automation of complex tasks:** Agentic systems can plan sequences of actions, interact with tools and environments (like browsing the web, using software APIs, managing databases), and iterate towards a defined goal. This is crucial for automating complex workflows that require more than just generating text or images.
3. **Increased capability and utility:** By allowing AI to *act* in the digital (and potentially physical) world based on its understanding, AI becomes far more useful for tasks like research, project management, data analysis, software development, and more.
4. **Current research focus:** There is significant research and development effort being put into building robust agentic architectures and frameworks. Concepts like reflection, memory, planning, and tool use are central to this effort.
5. **Early Examples:** While still in early stages and often experimental, examples like Auto-GPT, BabyAGI, and agent frameworks within libraries like LangChain demonstrate the potential for AI to carry out extended tasks with minimal human intervention after the initial goal is set.

However, it's important to note the challenges:

- * **Reliability:** Current agentic systems can be unpredictable, get stuck, or "go off track."
- * **Safety and Control:** Ensuring agents act safely and can be easily controlled or stopped is a major concern.
- * **Transparency:** Understanding *why* an agent took a certain action can be difficult.
- * **Ethics:** Issues around accountability, job displacement, and potential misuse are amplified with autonomous agents.

Conclusion:

Agentic AI is not just a theoretical concept; it's a very active area of development driven by the desire to make AI more capable and useful in automating complex tasks. While there are significant technical and ethical hurdles to overcome, the trend towards building AI systems that can perceive, plan, and act to achieve goals seems inevitable. So, yes, agentic capabilities will likely be a fundamental part of the future of AI, leading to more powerful and autonomous systems.

```
In [10]: response1 = model.generate_content("create table to explain about ai, generative ai")
print(response1.text)
```

Okay, here is a table explaining AI, Generative AI, and Agentic AI, highlighting the relationship and key characteristics.

Think of it this way:

- * **AI** is the *entire field*.
- * **Generative AI** is a *type* of AI focused on creation.
- * **Agentic AI** is a *paradigm* or *application* of AI focused on autonomous action towards a goal.

Here's the comparison table:

Comparison Table: AI, Generative AI, and Agentic AI

Feature	AI (Artificial Intelligence)	Generative AI
Scope	Broad Field: Encompasses any system mimicking human intelligence.	Subset of AI: Specifically focused on creating new content.
Paradigm/Application of AI	Focuses on autonomous goal-directed action.	Focuses on autonomous goal-directed action.
Primary Goal	To simulate, mimic, or perform tasks that typically require human intelligence (e.g., learning, problem-solving, perception, reasoning).	To generate new, original content (text, images, code, music, etc.) based on learned patterns.
Relationship	The overarching field. Generative AI and Agentic AI are parts or applications of AI.	A specific type within the broader field of AI. Can be a component used by Agentic AI.
Key Capability	Learning from data, pattern recognition, decision making, prediction, classification, problem-solving.	Creating novel data instances (generating new text, synthesizing images, composing music, writing code).
Typical Outputs	Decisions, predictions, classifications, recommended actions, processed data, insights.	New creative content (text, images, videos, audio, code, designs).
Complexity Focus	Varies widely, from simple automation to complex strategic games.	Primarily focused on generating a single, creative output based on a prompt.
Degree of Autonomy	Varies greatly depending on the application (supervised vs. unsupervised vs. reinforcement learning). Often requires human guidance or setup.	Relatively low in action. Typically requires a human prompt to trigger content generation. Doesn't inherently plan follow-up actions or goals.
Example Systems	Spam filters, recommendation engines, fraud detection systems, diagnostic tools, traditional chatbots, rule-based systems.	Large Language Models

ls (LLMs) like GPT-4, image generators like Midjourney/DALL-E, code generators like GitHub Copilot, AI music generators. | Autonomous AI agents (e.g., concepts behind Auto-GPT or BabyAGI), AI-powered personal assistants that manage schedules across apps, systems that automatically complete complex workflows based on high-level requests. |

****In Summary:****

- * ****AI**** is the foundation – the science of making machines smart.
- * ****Generative AI**** is about making machines **creative** – capable of producing new things.
- * ****Agentic AI**** is about making machines **proactive and goal-oriented** – capable of acting autonomously to achieve objectives, often by orchestrating various capabilities (which could include Generative AI).

```
In [11]: response2 = model.generate_content("create table to explain about List, Tuple, Set,
print(response2.text)
```


Okay, here is a table explaining the key characteristics of Python's List, Tuple, Set, and Dictionary data structures:

****Python Collection Types****

Feature	List	Tuple
Set	Dictionary	
Description Ordered, changeable collection. **collection.** **Unordered** , changeable collection of **unique** items. **Unordered** (insertion-ordered since 3.7+), changeable collection of **key-value** pairs.		
Syntax Defined using square brackets `[]`. Defined using parentheses `()`. Defined using curly braces `{}`. *Note: Empty set is `set()`, not `{}`.* Defined using curly braces `{}` with `key: value` pairs.		
Mutability **Mutable** (Can add, remove, change elements). **Immutable** (Cannot add, remove, or change elements after creation). **Mutable** (Can add or remove elements). **Mutable** (Can add, remove, or change key-value pairs).		
Order **Ordered** (Elements have a defined sequence/index). **Ordered** (Elements have a defined sequence/index). **Unordered** (Elements do not have a defined index or guaranteed order). **Ordered** (Elements maintain insertion order since Python 3.7+). Before 3.7 they were unordered.		
Duplicates **Allowed** (Can contain duplicate values). **Allowed** (Can contain duplicate values). **Not Allowed** (Automatically removes duplicate values). **Keys** must be unique. **Values** can be duplicates.		
Indexing/Access Accessed by **index** (e.g., `my_list[0]`). Accessed by **index** (e.g., `my_tuple[0]`). Cannot be accessed by index. Used for membership testing (`in`) or iteration. Accessed by **key** (e.g., `my_dict['key']`).		
Typical Use Cases Storing collections of items that might change (e.g., a list of user names to process, a shopping cart). Storing related pieces of data that should not change (e.g., coordinates `(x, y)`, RGB color `(r, g, b)`). Storing unique items, membership testing, mathematical set operations (union, intersection, difference). Storing data where you need to look up values based on a specific identifier (e.g., contact list `{'name': 'phone'}`, configuration settings).		

This table highlights the main differences in how each type handles order, mutability, duplicates, and access, which are the core considerations when choosing the right data structure for a task.

Translation

```
In [13]: response4 = model.generate_content("translate the response2 output into Hindi language")
          print(response4.text)                # need to give text to be translated
```

कृपया मुझे वह टेक्स्ट या सामग्री प्रदान करें जिसे आप "response2 output" कह रहे हैं और जिसका आप हिंदी में अनुवाद करवाना चाहते हैं।

मैं "response2 output" नाम के किसी विशिष्ट आउटपुट को नहीं देख सकता। जैसे ही आप मुझे वह टेक्स्ट देंगे, मैं उसका हिंदी में अनुवाद कर दूंगा।

```
In [14]: response5 = model.generate_content("translate following text into Hindi language: A  
print(response5.text)
```

ज़रूर, यहाँ उस पाठ का हिंदी अनुवाद दिया गया है:

एक अफ्रीकी परिदृश्य की अग्रभूमि में, एक हाथी प्रमुखता से खड़ा है, जो दर्शक की ओर मुंह किए हुए है। यह दृश्य सूर्योदय या सूर्यास्त की गर्म, जीवंत रोशनी में नहाया हुआ है, जिसमें सूरज फ्रेम के ऊपरी बाईं ओर दिखाई दे रहा है। आकाश में नारंगी और गुलाबी रंगों का एक सुंदर रंग-क्रम (gradient) दिख रहा है। हाथी के पीछे, एक जलाशय आकाश के ज्वलंत रंगों को प्रतिबिंबित कर रहा है। अग्रभूमि में ज़मीन रेतीली और सूखी दिख रही है, जहाँ कुछ विरल वनस्पति है। पृष्ठभूमि में पेड़ और थोड़ा उठा हुआ किनारा है, जो किसी नदी या पानी पीने की जगह (जलाशय) के किनारे का संकेत देता है। यह हाथी एक वयस्क है, जिसके दाँत स्पष्ट रूप से दिखाई दे रहे हैं और त्वचा खुरदरी है, और वह सूखी ज़मीन पर मज़बूती से खड़ा है। कुल मिलाकर, एक प्रभावशाली प्रकाश घटना के दौरान शांत प्रकृति का माहौल है।

Describe Image (multimodel)

```
In [16]: import PIL.Image # import Library to wo  
img = PIL.Image.open(r"C:\Users\Avinash\Downloads\elephant.jpg") # open image; as  
img
```

Out[16]:



```
In [17]: response3 = model.generate_content(img)  
print(response3.text)
```

An African elephant stands prominently in the center of the frame, facing forward with its head slightly turned to the right. Its large ears are spread, and its tusks are visible. The elephant is standing on a dry, sandy or dusty ground. Behind the elephant, there is a body of water reflecting the bright colors of a sunset or sunrise. The sky is a vibrant mix of orange and pink hues, with the sun appearing bright in the upper left portion of the image. Beyond the water, there are trees and vegetation on the horizon, silhouetted against the colorful sky. The overall scene captures a majestic animal in its natural habitat during a beautiful time of day.

Read text from image

```
In [37]: img1= PIL.Image.open(r"C:\Users\Avinash\Downloads\image with text.jpg")  
img1
```

Out[37]:



```
In [40]: response7 = model.generate_content(img1)  
print(response7)
```

```

response:
GenerateContentResponse(
  done=True,
  iterator=None,
  result=protos.GenerateContentResponse({
    "candidates": [
      {
        "content": {
          "parts": [
            {
              "text": "Here is the text visible on the signs in the image:\n\n1.
**BARBER SHOP**\n2. **DRAIN AND REFILL WITH Clean Clear GOLD TEXACO MOTOR OIL MANUF
ACTURED BY THE TEXAS COMPANY**\n3. **100% PURE PAR FIN BASE RPM MOTOR OIL REG. U.S.
PAT. OFF.**\n4. **COVER THE EARTH AFRICA EUROPE SHERWIN-WILLIAMS PAINTS** (Text on
this sign is heavily rusted and partially obscured)\n5. **DRUG STORE DRINK Coca-Col
a Delicious and Refreshing**\n6. **FINCK'S DETROIT-SPECIAL OVERALLS Wear Like a Pi
g's Nose TRY A PAIR The Man Who Thinks He's in FINCK'S**\n7. **PUBLIC TELEPHONE BEL
L SYSTEM**\n8. **IVORY SOAP Procter & Gamble Cincinnati, O.** (Some text is hard to
read)\n9. **Demand KAYO IT'S real CHOCOLATE**\n10. **GENUINE Ford MECHANIC ON THE P
REMISES**\n11. **WE SELL G COWHIDE BRAND OVERALLS & PANTS**\n12. **SHELL MOTOR OIL**
\n13. **Drink CANADA DRY Beverages**"
            }
          ],
          "role": "model"
        },
        "finish_reason": "STOP",
        "index": 0
      }
    ],
    "usage_metadata": {
      "prompt_token_count": 259,
      "candidates_token_count": 271,
      "total_token_count": 1161,
      "cached_content_token_count": 232
    },
    "model_version": "models/gemini-2.5-flash-preview-04-17"
  })
)

```

```

In [41]: response8 = model.generate_content(img1)
         print(response8.text)                                     # when we add .text it display

```

Here is the text visible in the image:

On the Barber Shop sign:

- BARBER SHOP

On the Texaco sign:

- DRAIN AND REFILL WITH
- Clean Clear GOLD.
- TEXACO MOTOR OIL
- MANUFACTURED BY THE TEXAS COMPANY

On the RPM sign:

- 100% PURE PAR FIN BASE
- RPM
- MOTOR OIL
- REG. U.S. PAT. OFF.

On the large Coca-Cola sign:

- DRUG STORE
- DRINK
- Coca-Cola
- TRADE MARK REG. U.S. PAT. OFF.
- Delicious and Refreshing

On the Finck's sign:

- FINCK'S
- "DETROIT-SPECIAL"
- OVERALLS
- Wear Like a Pigs Nose
- TRY A PAIR
- The Man who Thinks likes 'em is FINCK'S

On the Public Telephone sign:

- PUBLIC
- BELL SYSTEM
- TELEPHONE

On the Kayo sign:

- KAYO
- Demand
- Demand Kayo
- IT'S real CHOCOLATE

On the Genuine Ford sign:

- GENUINE
- Ford
- MECHANIC
- ON
- THE
- PREMISES

On the Ivory Soap sign:

- IVORY
- SOAP
- Procter & Gamble - 99 44/100 % Pure

On the Shell sign:

- SHELL
- MOTOR
- OIL

On the Cowhide Brand sign:

- WE SELL
- COWHIDE BRAND
- OVERALLS & PANTS

On the Canada Dry sign:

- Drink
- CANADA DRY
- Beverages

On the Sherwin-Williams sign:

- COVER THE
- EARTH
- AFRICA
- EUROPE
- SHERWIN-WILLIAMS
- PAINTS